Natural Language Processing: Project List and Problem Statements

Objective

The main objective is to investigate the impact of word embeddings, sentence embeddings, and their combination on software story point estimation. The study will compare the predictive performance of BERT-based word-level representations, SBERT-based sentence-level representations, and combined representations under different project settings and data-splitting strategies. All conclusions must be supported by reproducible experiments using Mean Absolute Error (MAE) and Median Absolute Error (MdAE).

TAWOS dataset link: https://doi.org/10.5522/04/21308124

1. Understanding the Project Concepts

1.1 What is software story point estimation?

Software effort estimation predicts the amount of work required to develop, modify, test, or maintain a software system. In agile development, story points are often assigned to issues or user stories to represent their relative effort or complexity. The goal is to build a machine learning model that uses a software issue description to predict its story point value.

Example issue description: “Fix the login failure when a user enters an incorrect password.”

The model predicts a story point value based on the target variable available in the dataset.

1.2 Text representation: word embeddings and sentence embeddings

Machine learning models require text to be represented numerically. Embeddings convert textual input into numerical vectors that capture lexical, contextual, or semantic information.

A. Word embeddings

- Word embeddings represent individual tokens as numerical vectors. Contextual token-level representations from BERT can capture information about words in their surrounding context.

B. Sentence embeddings

- Sentence embeddings represent an entire sentence or text passage as a numerical vector. SBERT produces sentence-level representations intended to capture semantic meaning.

C. Combined word and sentence embeddings

- This approach combines token-level information from BERT with sentence-level information from SBERT. The representations may be concatenated, projected into a common feature space, or fused using another clearly documented strategy.

1.3 Within-project versus cross-project estimation

A. Within-project estimation

- Training and testing issues come from the same software project. This setting measures performance on issues from a project represented in the training data, but the model may learn project-specific patterns.

B. Cross-project estimation

- Training data comes from one source projects, while evaluation is performed on a different target project. This setting examines how well representations generalise across projects.

1.4 Random splitting versus temporal ordering

A. Random split

- Issues are divided into training and testing sets without ordering them by creation date.

B. Temporal split

- Issues are sorted by creation date. Older issues are used for training,and the most recent issues for testing. Temporal validation must preserve chronological order.

2. Project List and Problem Statements

The 12 projects investigate the effect of text representation on story point estimation across within-project and cross-project settings, with random and temporally ordered data splitting.

Project	Text representation	Project setting	Data splitting	Problem statement
1	Word embeddings (BERT)	Within-project	Random split	Investigate the effectiveness of BERT-based word embeddings for story point estimation when training and testing data come from the same project using random splitting.
2	Word embeddings (BERT)	Within-project	Temporal ordering	Investigate the effectiveness of BERT-based word embeddings for predicting story points of later issues from the same project using chronological splitting.
3	Word embeddings (BERT)	Cross-project	Random split	Evaluate BERT-based word embeddings for story point estimation when training on source projects and testing on a different target project under the assigned random-split protocol.
4	Word embeddings (BERT)	Cross-project	Temporal ordering	Evaluate BERT-based word embeddings for predicting story points of later issues in a target project using source-project training data and chronological ordering.
5	Sentence embeddings (SBERT)	Within-project	Random split	Investigate the effectiveness of SBERT-based sentence embeddings for story point estimation within the same project using random splitting.
6	Sentence embeddings (SBERT)	Within-project	Temporal ordering	Investigate the effectiveness of SBERT-based sentence embeddings for predicting story points of later issues from the same project using chronological splitting.
7	Sentence embeddings (SBERT)	Cross-project	Random split	Evaluate SBERT-based sentence embeddings for story point estimation across different projects using the assigned random-split protocol.
8	Sentence embeddings (SBERT)	Cross-project	Temporal ordering	Evaluate SBERT-based sentence embeddings for predicting story points of later issues in a target project using chronological ordering.
9	Combined word and sentence embeddings	Within-project	Random split	Evaluate combination of BERT word embeddings and SBERT sentence embeddings on story point estimation within the same project.
10	Combined word and sentence embeddings	Within-project	Temporal ordering	Investigate the effectiveness of combined word and sentence embeddings for predicting story points of later issues within the same project.
11	Combined word and sentence embeddings	Cross-project	Random split	Evaluate whether combined word and sentence embeddings improve story point estimation when generalising from source projects to a different target project.
12	Combined word and sentence embeddings	Cross-project	Temporal ordering	Evaluate the effectiveness of combined word and sentence embeddings for predicting story points of later issues in a target project using chronological ordering.

3. Common Experimental Requirements and Expected Deliverables

3.1 Model architecture and text representation

- Use BERT for contextual token-level representations in word-embedding projects.
- Use SBERT for sentence-level representations in sentence-embedding projects.
- For combined-representation projects, combine BERT token-level representations with SBERT sentence-level representations using a clearly documented fusion strategy.
- A three-layer Convolutional Neural Network (CNN) may be used as the prediction component on top of the extracted representations.
- Document the model configuration, embedding extraction procedure, CNN architecture (if used), fusion strategy, hyperparameters, and random seeds.

3.2 Dataset splitting and five-fold validation

- Use an 80:20 development–testing split where compatible with the assigned project setting. Reserve 20% of the eligible data for final testing.
- Divide the 80% development portion into five folds. In each run, use four folds for training and one fold for validation, ensuring every fold serves as the validation fold once.
- Select the final model configuration using development data only. Evaluate the selected configuration on the held-out test set.
- Use the same data partitions when comparing word embeddings, sentence embeddings, and combined embeddings wherever a fair paired comparison is possible.
- For temporal projects, sort issues by creation date and use earlier issues for training and later issues for validation and testing.
- For cross-project projects, clearly document the source and target projects. Keep target-project test data separate from training and model-selection decisions, and follow the assigned project pairs.

3.3 Cross-project training and testing pairs

Keep thes**e 39 pairs** of training and testing data so that each project is considered as training data at least once:

Training Data	**Testing Data**
XD	MESOS
Usergrid	MXNet
Titanium Mobile Platform	Usergrid
The Titanium SDK	Alloy Framework
Software Cloud	Appcelerator Daemon
Sawtooth	Appcelerator Studio
Nexus	Command-Line Interface
MXNet	The Titanium SDK
Mule APIkit	Titanium Mobile Platform
Mule	Aptana Studio

Moodle	Atlassian Software Server
MongoDB Core Server	Bamboo
MESOS	Clover
Lyrasis Dura Cloud	Confluence Cloud
Lsstcorp Data management	Confluence Server
Jira Server	Crowd
Jira Cloud	FishEye
Java driver	Jira Cloud
Indy SDK	Jira Server
Indy Node	Software Cloud
FishEye	DotNetNuke Platform
Fabric	Blockchain Explorer
The MongoDB Engineering	Fabric
DotNetNuke Platform	Indy Node
DataCass	Indy SDK
Crowd	Sawtooth
Confluence Server	Lsstcorp Data management
Confluence Cloud	Lyrasis Dura Cloud
Compass	MongoDB Core Server
Command-Line Interface	Compass
Clover	C++ driver (MongoDB)
Aptana Studio	The MongoDB Engineering
C++ driver (MongoDB)	Java driver
Blockchain Explorer	Moodle
Bamboo	Mule
Atlassian Software Server	Mule APIkit
Appcelerator Studio	Nexus
Appcelerator Daemon	DataCass

Alloy Framework	XD

3.4 Evaluation metrics and required results

Each group must report at least the following metrics:

- Mean Absolute Error (MAE)
- Median Absolute Error (MdAE)
- Five-fold validation results, reported separately from final held-out test results
- Where feasible, mean and standard deviation of validation metrics across folds

Representation	Model configuration	MAE	MdAE
BERT word embeddings	Report model and settings	Report	Report
SBERT sentence embeddings	Report model and settings	Report	Report
Combined BERT + SBERT embeddings	Report model and settings	Report	Report

Each group must discuss:

- Which representation achieves the lowest MAE and MdAE.
- Limitations, threats to validity, and possible explanations for the observed results.

3.5 GitHub repository and code submission

Each group must create a GitHub repository containing the complete implementation and documentation. The repository must include:

- README.md with project title, group members, problem statement, objectives, dataset description, setup instructions, and instructions for reproducing experiments.
- Source code or notebooks for preprocessing, embedding extraction, model training, prediction, and evaluation.
- Configuration files specifying model names, hyperparameters, random seeds, split settings, and embedding-fusion settings where applicable.
- Results containing validation and test metrics (MAE and MdAE), comparative tables, and appropriate plots.

3.6 Final deliverables

- GitHub repository link containing the complete source code and documentation.
- Final project report in PDF format describing the problem, methodology, experiments, results, conclusions, and the GitHub link on the first page.
- Comparative evaluation tables reporting MAE and MdAE for the assigned representation(s).
- Five-fold validation results and final held-out test results.
- Demonstration of the implementation, including instructions to run the code and reproduce the principal results.

3.7 General guidelines

- Follow the assigned project configuration and common evaluation metrics.
- Use training data for fitting model parameters and validation data for model selection. Reserve the test set for final evaluation.
- Record random seeds, data-splitting procedures, model versions, and experimental configurations.
- Report actual experimental findings; do not fabricate results or claim improvements unsupported by measurements.
- Ensure that all group members understand the methodology, code, results, and conclusions and can explain them during the final evaluation.